AI Mock Interview and Personalised Learning Assistant: System Report

SkillBridge (Academic-to-Industry Skill Bridge), project J26-SE-328-RP

Author: Dilmith Ranasinghe · **Date:** 18 October 2026 · **Code state:** `main` plus uncommitted work (section 11)

1. Summary

The component prepares students for industry roles in two ways:

1. **AI Mock Interview Assistant** (`services/interview-service`). A video interviewer asks role- and level-specific technical questions, listens to the spoken answer, grades it against a reference answer and rubric, and gives spoken feedback and an end-of-interview report.
2. **Personalised Learning Assistant** (`services/learning-service`). The student asks a technical question and gets an answer grounded in official documentation, with citations and a recommended next topic.

Both parts run **fully on a local machine with no cloud API key**. **No model was trained or fine-tuned**; all models are pre-trained and used for inference (section 7).

Measure	Value
Job roles	4: Backend Developer, Full-Stack Developer, DevOps Engineer, Database Engineer
Experience levels	3: Entry, Mid, Senior
Interview questions	**223** (28 curated + 195 from public sources), each with a reference answer and rubric
Learning documents	**1,066** documents, **21,376** searchable passages, from 13 public sources plus the curated curriculum
Evaluator agreement with human graders	Quadratic weighted kappa **0.62**, Pearson **0.63** (Mohler dataset, 78 answers)

2. Job roles

Role	Interview questions (Entry / Mid / Senior)	Learning documents available
DevOps Engineer	**72** (24 / 25 / 23)	704
Backend Developer	**55** (14 / 13 / 28)	497
Full-Stack Developer	**55** (5 / 45 / 5)	308
Database Engineer	**41** (13 / 15 / 13)	99

A learning document can serve several roles (for example the MDN HTTP guides serve both Backend and Full-Stack), so the learning totals overlap. Git and pull-request documents (32) are available to every role.

3. Interview question bank

3.1 Where the questions come from

Source	Questions	Licence	How it was used
test-your-sysadmin-skills	40	MIT	DevOps questions with community answers and Junior / Regular / Senior levels
reactjs-interview-questions	30	MIT	Full-Stack questions with answers
Curated technical interview bank	28	Project	The original hand-written rubric questions
The System Design Primer	15	CC BY 4.0	Worked design solutions, turned into focused spoken questions
Front-end Developer Interview Questions (h5bp)	9	MIT	Full-Stack questions; answers written from MDN
Written from documentation (section 4)	101	Per source	PostgreSQL 23, MDN 16, GitHub Docs 12, SQLite 10, Azure 10, Kubernetes 8, Generative AI 6, Prometheus 6, AWS guide 4, Docker 4, curriculum 2

No public dataset of technical interview questions with model answers and rubrics exists, and none covers Database Engineer, so database questions were written from the PostgreSQL and SQLite documentation.

3.2 Topics by role and level

Backend Developer

- *Entry:* API design and protocols, authentication and authorisation, connection pooling, HTTP request headers, idempotent consumers, competing consumers, ambassador pattern, function calling, retrieval-augmented generation, version control, supply-chain security.
- *Mid:* distributed caching, HTTP caching and connection models, microservices, anti-corruption layer, data partitioning, database indexing, transient fault handling, semantic search, responsible AI, Git remotes.
- *Senior:* system design (URL shortener ID generation, Twitter timeline fan-out, social-graph search, sales rank, web-crawler scaling, multi-data-centre scaling, LRU cache), distributed systems, event-driven systems, fault-tolerance and gateway routing patterns, rate limiting, CDNs, content security policy, LLM lifecycle and model comparison.

Database Engineer

- *Entry:* normalisation, relational modelling, constraints, query plans and planner statistics, partial and index-only scans, bulk loading, replication methods, window functions, SQLite crash recovery.
- *Mid:* transaction isolation and durability, concurrency, index structures, SQL optimisation, foreign keys, schemas, vacuuming, row-level security, privileges, point-in-time recovery.
- *Senior:* MVCC and concurrency control, index types and maintenance, PostgreSQL replication and hot standby, recursive CTEs, JSON storage, scalable database architectures, SQLite transaction modes.

DevOps Engineer

- *Entry:* Linux commands and permissions, shell processes and signals, SSH, DNS records, packet filtering, containers, Kubernetes networking and storage classes, CI, Prometheus basics, EC2 instance types.
- *Mid:* container orchestration and optimisation, Kubernetes logging and storage scheduling, the Linux boot process, LVM, NAT, network troubleshooting, log management, resource limits, SSH key security, S3 consistency, CI/CD migration.
- *Senior:* site reliability engineering, cloud infrastructure and scaling bottlenecks, Kubernetes deployments and gang scheduling, memory management, DNS resolution, password hashing, shell scripting security, Prometheus data model and remote-write tuning, AMI build strategy, Varnish caching.

Full-Stack Developer

- *Entry:* frontend performance and browser metrics, web security, JavaScript control flow, Git.
- *Mid:* React (hooks, lifecycle, rendering and diffing, portals, routing, higher-order components, React 19), Redux and state management, JavaScript types and collections, code splitting, rendering patterns and pipeline, same-origin policy, server rendering.
- *Senior:* application security, web security defences, the critical rendering path, JavaScript object properties, Git ignore strategies.

3.3 How the bank was built

1. **Collect candidates** (`scripts/datasets/extract_interview_candidates.py`): 245 candidates, either questions from the open sets (with their own answers where they exist) or documentation pages to write a question from.
2. **Ground and write** (`generate_interview_bank.py`, local Gemma 4 E4B): for each candidate the closest learning-corpus passages are retrieved. The model must answer **only from the source answer and those passages**. It keeps or rejects the question and writes the spoken question, a 3–6 sentence reference answer, 3–4 rubric criteria, and descriptions of a strong and a weak answer.
3. **Quality controls:**
    - **Rejections:** 48 of 245 candidates were rejected, for example opinion questions ("your favourite CLI tools"), trivia, or questions the material doesn't support.

- **Output checks:** items that refer to "the passage" or "the provided text", or that lack a valid answer or rubric, are dropped.
- **Levels:** the source's level is kept where one exists, because the model labelled almost everything "Mid Level".

4. **Provenance:** every item records its source, licence, URL and the documents that ground its answer.

4. Learning database: modules

Each module is a set of documents with one topic. Counts are documents.

4.1 Backend Developer

Source	Modules
SkillBridge curriculum (10)	REST principles, gRPC vs REST, JWT authentication, rate limiting, connection pooling, database indexing, Redis caching, Kafka and RabbitMQ, circuit breakers, distributed tracing
MDN Web Docs (201, shared with Full-Stack)	HTTP guides (45), HTTP headers (39), HTTP methods (11), HTTP status codes (60), web security (46)
Azure Architecture Center (74, shared with DevOps)	Cloud design patterns, architecture styles, cloud best practices, microservices architecture
The System Design Primer (26, shared with DevOps)	System design concepts, system design case studies
GitHub Docs (78, shared with DevOps)	GitHub Actions CI/CD
Generative AI for Beginners (15, shared with Full-Stack)	Generative AI and RAG: prompt engineering, chat and search applications, function calling, security, RAG and vector databases, agents, fine-tuning
PostgreSQL and SQLite (shared with Database)	See 4.4

4.2 Full-Stack Developer

Source	Modules
SkillBridge curriculum (7)	React and Next.js rendering patterns, state management, micro-frontends, WebSockets, Web Vitals, accessible design, frontend security (OWASP)
MDN Web Docs (53)	JavaScript guide (33), web performance (15), Fetch API (3), Web Storage API (2)
Shared	MDN HTTP and web security (201), Generative AI and RAG (15), Git (32)

4.3 DevOps Engineer

Source	Modules
SkillBridge curriculum (6)	CI/CD with GitHub Actions, Terraform, Kubernetes pods, Docker multi-stage builds, cloud VPC security, Linux performance
Kubernetes Documentation (172)	Workloads (38), cluster administration, scheduling and eviction, security, storage, overview, services and networking, architecture, resource management, configuration, containers, policy
Docker Documentation (138)	Getting started, build (building, cache, BuildKit, concepts), Compose (intro, how-tos), engine (containers, network, storage, logging, security, resource limits)
The Open Guide to AWS (114)	AWS services: EC2, S3, IAM, networking, load balancers, API Gateway, AMIs and more, with tips and gotchas
Prometheus Documentation (44)	Monitoring: data model, metric types, instrumentation, alerting, practices
Kubernetes the Hard Way (12)	Building a cluster by hand
The Art of Command Line (8)	Linux command line
Shared	Azure architecture (74), system design (26), GitHub Actions (78), Git (32)

4.4 Database Engineer

Source	Modules
SkillBridge curriculum (6)	Isolation levels and ACID, EXPLAIN ANALYZE, B-tree / GIN / BRIN indexes, partitioning, sharding and replication, NoSQL vs relational
PostgreSQL 18 Documentation (45, shared with Backend)	Indexes (10), SQL queries (8), concurrency control (5), data definition (5), performance (5), maintenance and WAL (4), replication and high availability (4), backup and restore (2), security (2)
SQLite Documentation (25, shared with Backend)	Query planner, transactions and isolation, WAL, atomic commit, locking, data types, foreign keys, partial and expression indexes, window functions, JSON, upsert, table types
Shared	Git (32)

5. How the interview works

1. **Sign in.** The page requires a logged-in account; the candidate's name and ID come from the session, nothing is hard-coded.
2. **Upload a CV** (PDF, DOCX or TXT, read in memory and not stored). The LLM extracts skills, a suggested role, a level and a one-line background. Stated facts (years of experience, student status) override the model's level guess. There is a keyword fallback and a "continue without a CV" option.
3. **Choose role and level** from dropdowns (pre-selected from the CV) and optionally consent to camera-based delivery analysis (off by default).
4. **Question selection:**
   - **Search:** the service searches the bank for the role (vector similarity on role, level and skills) and takes the 60 closest items.
   - **Filters:** it removes questions the student had in earlier sessions and topics already asked in this one, and prefers the chosen level.
   - **Pick:** it chooses at random among the best 6 matches, so sessions vary.
   - **Phrasing:** the LLM rewrites the bank question as a natural spoken question, and follows up on concepts the previous answer missed.
5. **The interviewer speaks.** The local avatar engine renders the question as video: LivePortrait for head motion and blinking, MuseTalk for lip-sync, and the macOS text-to-speech voice. Text is streamed sentence by sentence, so the first sentence plays in about 11–18 s, and the next question is prefetched during feedback.
6. **The candidate answers.** Controls: camera on/off, microphone on/off (muting pauses the recording) and Submit Answer. Answers are recorded in the browser, with silence measured for the pause ratio.
7. **Speech-to-text:** faster-whisper (small.en) transcribes the answer in the container (about 6 s for 20 s of speech).
8. **Evaluation:**
   - **Inputs:** the LLM grades the transcript against the bank item's **reference answer and rubric** criteria.
   - **Scores:** it scores four criteria from 1 to 5 (technical accuracy, relevance, explanation quality, clarity) and lists strengths, gaps and missing concepts. The overall score is 70% technical accuracy, 15% clarity and 15% delivery.
   - **Spoken reaction:** the interviewer says a short reaction aloud.
9. **Delivery cues (with consent only):**
   - **Camera cues:** MediaPipe Face Landmarker runs in the browser and measures observable behaviour only: eye contact relative to the candidate's own head pose, face visibility, smiling, brow tension, blink rate and head steadiness.
   - **Speech cues:** pause ratio, filler words and speaking pace come from the audio and the transcript.
   - **No emotion inference:** mood and emotion are deliberately not inferred. Such inference is unreliable, and the EU AI Act (Article 5) prohibits emotion recognition in education.
10. **End interview:** a report with average scores, a delivery summary and per-question feedback; unanswered questions are excluded.

6. How the learning assistant works

1. **Question:** typed, or chosen from suggestions built from the curriculum for the student's role (one per topic, preferring topics that match CV skills).
2. **Hybrid retrieval** over pgvector:
   - **Dense search:** bge-small embeddings (384 dimensions) with HNSW cosine search.
   - **Sparse search:** PostgreSQL full-text ranking (`ts_rank_cd`).
   - **Fusion:** results are merged with Reciprocal Rank Fusion into about 8 candidates, filtered by role.
3. **Re-ranking:** a rule-based scorer (term coverage, phrase match, title and topic affinity, blended with vector similarity) keeps the best 3.
4. **Context compression:** boilerplate is removed and the passages are ordered to avoid the "lost in the middle" effect.
5. **Grounded generation:** the local LLM answers using only the passages, cites them, and proposes a next topic.
6. **Response:** the answer, source citations (with licence), latencies, and a "learn this next" link. The query and topic are logged as progress.
7. **Errors are shown, not hidden.** A failed request shows the real error. It no longer shows a hard-coded sample answer.

7. Models

Nothing was trained or fine-tuned. All models are pre-trained by their publishers and used for inference.

Model	Category	Architecture	Used for	Runs on
Gemma 4 E4B (Google), 4-bit MLX	Deep learning, LLM	Decoder-only transformer	Questions, evaluation, feedback, CV profile, learning answers, building the question bank	Mac GPU
bge-small-en-v1.5 (BAAI)	Deep learning, embedding	BERT-based bi-encoder, 384-d, ONNX	Retrieval for both the interview bank and the learning corpus	Container CPU
Whisper small.en (OpenAI), faster-whisper int8	Deep learning, speech recognition	Encoder–decoder transformer	Transcribing answers	Container CPU
MediaPipe Face Landmarker (Google)	Deep learning, vision	Face mesh and blendshape networks	Delivery cues, with consent	Browser (WebAssembly or GPU)
LivePortrait (Kuaishou)	Deep learning, generative vision	Keypoint motion and warping network	Interviewer head motion and blinking	Mac GPU
MuseTalk 1.5 (Tencent Music), MLX port	Deep learning, generative vision	Latent inpainting UNet with Whisper audio features	Lip-sync	Mac GPU
TAESD	Deep learning, autoencoder	Small convolutional decoder	Fast image decoding for MuseTalk	Mac GPU
macOS speech synthesis	Text-to-speech	System voice	Interviewer voice	Mac

Gemini and OpenAI remain optional cloud providers. Non-neural techniques: PostgreSQL full-text ranking, Reciprocal Rank Fusion, the rule-based re-ranker and compression, the HNSW approximate nearest-neighbour index, and keyword matching as a CV fallback.

8. Evaluation and measurements

8.1 Interview evaluator against human graders (Mohler ASAG)

Dataset: 2,273 computer-science student answers, each scored 0–5 by two human graders (CC BY 4.0). The run used 78 answers sampled evenly across four human-score bands, and the evaluator's technical-accuracy score was compared with the graders' average.

Measure	Question and answer only	With reference answer (live setup)
Pearson correlation	0.56	**0.63**
Quadratic weighted kappa	0.49	**0.62**
Mean absolute error (0–5)	0.92	**0.81**

- **Reference answers help.** Giving the evaluator the reference answer improved every measure, which is why live interviews now grade against the bank's reference answer and rubric.
- **The human graders disagree with each other.** Grader 1 averaged 4.5 and grader 2 averaged 3.0, and they agree at only r = 0.48. Against the individual graders the evaluator scores r = 0.52 (grader 2) and 0.18 (grader 1, who gave nearly every answer 4 or 5).
- **Scores are compressed toward the middle:**

Human score band	Human average	Model average
Strongest answers	4.8	3.8
Weakest answers	1.5	2.1

- **Failures:** 3 of 160 gradings failed and were excluded.
- **Sample size:** with 78 answers the correlation gain is within the margin of error; the kappa gain is larger.

8.2 Learning assistant (live, full corpus)

Role	Question	Cited	Search time	Answer time
Database	How does MVCC avoid read locks?	PostgreSQL Explicit Locking and Transaction Isolation, curriculum isolation document	452 ms	49 s (model warming up after idle)
DevOps	How do Prometheus histograms work?	Prometheus "Histograms and summaries", "Metric types"	483 ms	12 s
Backend	When should I use the circuit breaker pattern?	Azure "Circuit Breaker pattern"	667 ms	10 s
Full-Stack	What is the critical rendering path?	MDN "Critical rendering path"	473 ms	8 s

8.3 Interview timings (local stack)

Step	Time
Question text generated	about 2–11 s
Interviewer starts speaking	about 11–18 s
Speech-to-text (20 s of speech)	about 6 s
Evaluation	about 12–18 s

8.4 Automated tests

Suite	Result
interview-service (35 tests)	**31 pass.** One end-to-end test needs a database on `localhost:5432`. Three speech-to-text tests assume no local Whisper is installed, which is not true inside the container. None relate to the work in this report.
learning-service (6 tests)	**4 pass**; the 2 end-to-end tests have hard-coded `localhost` database credentials
Frontend	TypeScript type check and lint clean for the changed files

9. Data pipeline and reproducibility

All scripts are in `scripts/datasets/`; raw downloads are kept outside the repository on the external drive.

Step	Script	What it does
Download	`fetch_datasets.sh`	Shallow, sparse Git clones of the documentation folders; PostgreSQL and SQLite HTML pages; Mohler parquet
Learning corpus	`build_learning_corpus.py`	Converts each source to clean Markdown with role, topic, source, licence and URL headers; drops pages under 600 characters; writes `ATTRIBUTION.md`
Load	learning-service startup	Embeds any documents the database doesn't have yet, in the background; saves per document, so an interrupted load resumes
Interview bank	`build_interview_bank.sh`	Section 3.3; resumable; writes `services/interview-service/data/public_question_bank.json`
Evaluator check	`run_mohler_eval.sh`	Section 8.1; results saved as JSON

10. Licences

Licence	Sources
CC BY 4.0	Kubernetes docs, Azure Architecture Center, The Open Guide to AWS, The System Design Primer, GitHub Docs, Mohler ASAG
CC BY-SA (share-alike)	MDN Web Docs (2.5), The Art of Command Line (4.0)
Apache-2.0	Docker docs, Prometheus docs, Kubernetes the Hard Way
MIT	Generative AI for Beginners, test-your-sysadmin-skills, reactjs-interview-questions, h5bp interview questions
PostgreSQL Licence	PostgreSQL 18 documentation
Public domain	SQLite documentation

Every document and question records its source and licence. Share-alike material must keep its licence if the processed files are redistributed. Rejected sources: ByteByteGo system-design-101 (no derivatives), the Google SRE book (no derivatives), Pro Git (non-commercial), Stack Overflow dumps (download terms forbid AI use), and Hugging Face "interview" datasets (off-topic or template answers).

11. Work status

All of the following is **uncommitted** (nothing has been pushed):

- **Login-based user:** the mock interview uses the logged-in user and the route is protected.
- **Learning service fixes:** the startup import fix (the corpus had never loaded), the local LLM and embeddings, the dimension-change rebuild, memory-safe resumable background ingestion, real errors instead of a fake answer, and suggested questions by role.
- **Public data:** dataset scripts, the 1,037-document external corpus and the 195-question public bank.
- **Interview evaluation and variety:** grading against reference answers and rubrics (new turn columns), varied question selection, and the Mohler evaluation script.

12. Limitations and future work

1. **Fixed benchmark figures:** `rag_evaluator.py` and the Viva Inspector still show fixed, pre-written benchmark values. They must not be reported as results until a real run over a test set replaces them.
2. **Out-of-date test set:** the 40-question learning test set was written for the original 29 documents and needs extending for the new corpus.
3. **Full-Stack levels:** 45 of 55 questions are Mid level, because the React and h5bp sets give no levels.
4. **Model-written reference answers:** they are grounded in cited material but should be spot-checked before use in real assessment. At least one item (GitHub coding agents) tests product details rather than understanding.
5. **Score compression:** the evaluator is hard on excellent answers and soft on weak ones. Calibration (example answers per score, or a mapping fitted to human scores) is the next step.
6. **Hard-coded learning priority:** "Database Connection Management & Scalability" is fixed and should come from the Skill Analysis component.

7. **Speed:** answers take 8–15 s and searches about 0.5 s on 21,000 passages; a full-text index and a neural re-ranker (for example `bge-reranker-base`) are the planned improvements.
8. **First load time:** loading the full corpus takes 15–25 minutes on first start.

13. How to run

```
# Host Mac (Huggingbay folder)
./start-avatar.sh          # avatar engine, port 8768
./start-llm.sh             # Gemma 4 E4B, port 8082

# Project folder (J26-SE-328-RP)
docker compose -p skill-bridge -f infrastructure/docker-compose.yml \
  -f ~/.skillbridge-local/docker-compose.override.yml up -d
cd frontend && npm run dev # http://localhost:3000/mock-interview

# Rebuild the public data (optional)
scripts/datasets/fetch_datasets.sh
"/Volumes/Dilmith Ext/datasets/.venv/bin/python" scripts/datasets/build_learning_corpus.py
scripts/datasets/build_interview_bank.sh
scripts/datasets/run_mohler_eval.sh --limit 80
```